

Operator Learning in Heterogeneous Agent Models

Yaolang Zhong*

July 2026

Abstract

This paper develops a solution framework for dynamic macroeconomic models with a continuum of heterogeneous agents and aggregate uncertainty, in which an agent's state includes the cross-sectional distribution of all agents, an infinite-dimensional object. Existing methods first compress this distribution into a finite, modeler-chosen object (a set of moments, a panel of simulated agents, a histogram, or a fixed number of learned moments), which caps accuracy before solving begins. I instead reformulate the policy as an operator that conditions on the distribution as a function, without fixing in advance a finite summary of it; the method learns its own summary in a spectral basis whose resolution is refinable and independent of the discretization of the distribution. I parameterize the operator with a Fourier neural operator. I evaluate the framework in four economies: (i) the canonical [Krusell and Smith \(1998\)](#) model; (ii) a version with a continuous idiosyncratic shock; (iii) a two-asset HANK model; and (iv) a decreasing-returns firm model in which the wage depends on the dispersion of the firm distribution. In the first two, approximate aggregation holds: the operator attains a relative Euler error below one percent in minutes and simulates stably where a parameter-matched feed-forward network diverges, though at comparable accuracy it only matches a well-specified moment method. In the last two, an equilibrium price is a nonlinear functional of the distribution's shape and approximate aggregation fails; there, conditioning on the whole distribution recovers the shape-dependent price that moment truncation and moment-bottleneck networks miss. The framework's value is therefore not uniformly higher accuracy, but a structured and assumption-light way to carry the distribution as a genuine state variable: it renders the distributional state tractable and generalizable, it learns a sufficient distributional summary without hand-picking moments, and, by conditioning on the whole distribution rather than a fixed moment set, it is less exposed to the mis-specification a fixed moment rule suffers when a price depends on the distribution's shape.

Keywords: Artificial intelligence, Machine learning, Deep learning, Neural network, Dynamic models, Dynamic programming, Euler equation

JEL: C61, C63, C65, C68, C88, E32, E37

*Graduate School of Economics, University of Tokyo. Email: yaolang.zhong@gmail.com.

1 Introduction

Dynamic models that incorporate both heterogeneity and aggregate uncertainty have become central to macroeconomics. Heterogeneity shapes aggregate consumption, savings, and labor supply (Blundell, Pistaferri and Preston, 2008; Krueger and Perri, 2006), plays a pivotal role in the propagation of aggregate shocks (Kaplan, Moll and Violante, 2018), and yields more realistic representations of the economy (Cagetti and De Nardi, 2008). A parallel reason for their prevalence is the development of numerical methods capable of solving them. These models are notoriously hard to solve, but advances in computation, and most recently the application of deep learning (Maliar, Maliar and Winant, 2021; Maliar and Maliar, 2022; Han and Yang, 2021; Azinovic, Gaegauf and Scheidegger, 2022), have made global solutions attainable, exploiting the approximation power of neural networks and simulation-based data generation to mitigate the curse of dimensionality.

The defining difficulty of this class of models is that an agent’s state comprises her own individual vector together with the cross-sectional distribution of all agents’ states, an infinite-dimensional object. Solving for the policy $\mathbf{g}(k, \epsilon, Z; \mathbf{\Gamma})$ therefore means learning a function whose argument is itself a function. The central question is how to represent and carry that distribution. This paper’s answer is a reformulation of the policy as an operator, a mapping from functions to functions. In this form the state is processed in two stages. An operator $\mathbf{G}$ takes only the distribution $\mathbf{\Gamma}$ as input and returns a conditional policy function, which is then evaluated at the individual state (k, ϵ, Z) . Solving the policy thus becomes learning the operator $\mathbf{G}$, and I parameterize it with a neural operator, a machine-learning architecture designed to approximate mappings between function spaces (Li et al., 2020b; Kovachki et al., 2021), whose approximation guarantees trace back to the universal approximation theorem for operators (Chen and Chen, 1995).

Conditioning on the distribution as a function. To see what this reformulation buys, contrast it with the two prevailing frameworks. The first, the Krusell–Smith framework (Krusell and Smith, 1998; Den Haan and Rendahl, 2010; Maliar and Maliar, 2014), hereafter KS, tracks a small, fixed set of moments of $\mathbf{\Gamma}$ and posits a law of motion for them. It is tractable, intuitive, and fast, and where a few moments forecast prices well, the approximate-aggregation result of Den Haan and Rendahl (2010), it is highly accurate. It nonetheless imposes a bounded-rationality assumption, in that agents perceive only the moments, and an ad hoc choice of which moments to track, a choice whose sufficiency is case-dependent. The second, the deep-learning or feed-forward framework (Maliar, Maliar and Winant, 2021; Han and Yang, 2021; Azinovic, Gaegauf and Scheidegger, 2022; Kahou, Fernández-Villaverde, Perla and Sood, 2021), hereafter NN, keeps the full distribution but represents it by a finite object fed to a network: a panel of N simulated agents, a grid histogram, or a fixed number of learned generalized moments. This lifts the moment restriction but fixes a different bottleneck.

The common thread is that every existing method first compresses Γ into a finite, modeler-chosen representation (how many moments, how many agents, how fine a grid) before learning begins. Each such choice caps the distributional information the policy can use, an accuracy ceiling set a priori, and ties the solution to that resolution, so that refining it requires retraining. The operator reformulation moves this bottleneck off the input. Conditioning on Γ as an element of a function space fixes no finite summary of the distribution in advance; the method learns its own summary from the data, in a spectral basis whose resolution is a refinable network hyperparameter rather than a pre-committed count of moments or agents. This is the paper’s central claim.

Structural properties as consequences. Three properties often emphasized for this class of methods follow automatically from the operator formulation: discretization invariance (the network size is independent of the number of simulated agents, so it need not be retrained when the discretization of Γ changes, a property [Kovachki et al. \(2021\)](#) characterize as essentially defining the neural-operator class), permutation invariance (the policy is invariant to the ordering of agents), and aggregation sharing (the shared aggregate is processed once rather than redundantly for each agent). They follow because the formulation conditions on the empirical distribution function rather than on a raw agent vector. These properties are best read as desiderata rather than as this paper’s distinguishing claim: the KS framework already possesses all three, and recent symmetric (deep-sets) networks attain permutation invariance ([Han and Yang, 2021](#); [Kahou et al., 2021](#)). What remains exclusive to the operator is the bottleneck-free, function-space conditioning, of which the three properties are consequences. Table 1 summarizes the comparison.

Table 1: Comparison of Three Numerical Frameworks for the Desirable Properties

Property	Framework		
	KS ¹	NN ²	Operator ³
Full Information of Distribution	×	✓	✓
Discretization-Invariance	✓	×	✓
Permutation-Invariance	✓	×	✓
Sharing-Aggregation	✓	×	✓

¹ Krusell-Smith

² Deep Learning with feed-forward neural network

³ Deep Learning with neural operator (This Paper)

What the framework delivers. I develop a tailored optimization strategy, comprising a hybrid sampling scheme (grids for the low-dimensional individual state, stochastic simulation over the ergodic set for the infinite-dimensional distribution, following [Judd, Maliar and Maliar, 2011](#)), a pre-training initialization against a static no-aggregate-shock solution, and

Euler-residual training with a target network and standard stabilizers. I evaluate the framework in four economies, deliberately including cases in which it is not expected to win. On the canonical [Krusell and Smith \(1998\)](#) economy the operator attains a relative Euler error below one percent in minutes, an exacting standard for a policy with a borrowing-constraint kink ([Maliar, Maliar and Winant, 2021](#)), and keeps the simulated economy on a consistent ergodic path; a parameter-matched feed-forward network driven by the raw cross-section instead diverges in simulation in nine of ten seeds, which identifies the operator’s structured representation as the source of its stability (Section [6.1](#)). A second economy replaces the binary shock with a continuous idiosyncratic productivity, showing that the reformulation carries a genuinely two-dimensional joint distribution at comparable accuracy. These two are validation exercises: approximate aggregation holds, the shape of the distribution is nearly static, and a well-specified moment method already suffices. The operator’s distinctive value appears in two economies where approximate aggregation fails because an equilibrium price is a functional of the distribution’s shape: a two-asset HANK economy in which the liquid return depends on the joint shape of wealth, and a decreasing-returns firm economy in which the wage depends on the dispersion of the firm distribution. In both, conditioning on the whole distribution recovers the shape-dependent price that moment truncation and moment-bottleneck networks do not. The detailed findings appear in Section [6](#).

Contributions to the Literature: This paper contributes to computational macroeconomics a new solution framework for heterogeneous-agent models with aggregate uncertainty. The principal challenge is learning a policy whose input includes a distribution function. Every existing approach first reduces the infinite-dimensional distribution to a finite, modeler-chosen object before learning: a low-dimensional vector of hand-picked moments ([Krusell and Smith, 1998](#); [Den Haan and Rendahl, 2010](#); [Maliar and Maliar, 2014](#)), a high-dimensional panel of simulated agents ([Maliar, Maliar and Winant, 2021](#)), a grid histogram ([Azinovic, Gaegauf and Scheidegger, 2022](#)), or a fixed number of learned generalized moments ([Han and Yang, 2021](#); [Kahou, Fernández-Villaverde, Perla and Sood, 2021](#)). Each such choice caps attainable accuracy before solving begins. The central innovation is conceptual: I reformulate the policy as an operator conditioning on the distribution as a function, so that no modeler-imposed bottleneck is fixed in advance; the method learns its own summary, refinable at any resolution. Parameterizing this operator with a Fourier neural operator, together with the tailored optimization strategy, serves that reformulation. A second, empirical contribution is a deliberately demanding, multi-method evaluation across four economies that documents where the operator helps (rendering the distributional state tractable and generalizable, learning a sufficient distributional summary without hand-picking moments, and conditioning on the whole distribution rather than a fixed moment set) and where it only matches a well-specified moment method on moment-friendly ergodic sets. This explicit delineation of scope is itself a contribution, in a literature that tends to emphasize favorable comparisons.

While the neural operator is a tool from the machine-learning literature ([Lu et al., 2019](#); [Kovachki et al., 2021](#); [Li et al., 2020b,a, 2021](#); [Goswami et al., 2023](#)), this paper also con-

tributes back to it. Neural operators have to date been applied chiefly to partial differential equations characterizing physical systems. To the best of my knowledge this is their first application to discrete-time economic models, and it is not a mere transplant: formulating the economic problem so that the operator’s domain knowledge, an integral aggregation from individual to aggregate, is exploited is itself part of the contribution. The framework also connects to the emerging literature on strategic interaction among reinforcement-learning agents in economic environments (Chen et al., 2021; Zheng et al., 2020, 2021), where the operator formulation and neural-operator architecture offer directions for policy-optimization algorithms.

The remainder of the paper is structured as follows. Section 2 introduces the economic environment; Section 3 contrasts my computational strategy with the KS and NN frameworks; Section 4 presents the neural operator and its properties; Section 5 covers implementation; Section 6 reports the results across the four economies; and Section 7 concludes.

2 The Economic Environment

This section provides a brief overview of the economic environment from Den Haan, Judd and Juillard (2010), a Bewley-Huggett-Aiyagari model¹ with a continuum of agents, aggregate uncertainty, idiosyncratic shocks, and an incomplete market, which I intend to address using my method. This model serves as a benchmark for the comparison of the properties of numerical algorithms. It is the leading example on which the method is developed in Sections 3–5; Section 6 then evaluates the framework on it and on three further economies, each a modification of the individual state or the equilibrium price to which the same operator-learning procedure applies.

For clarity from this point forward, I will use lowercase letters for individual variables, uppercase letters for aggregate variables, bold lowercase letters for functions and bold uppercase letters for operators.

Agents’ Problem

There is a continuum of infinitely lived and ex-ante identical agents. Each period the agents receive the time endowment $\bar{l}$ and face a binary income shock ϵ : agents are employed when $\epsilon = 1$, and earn the after-tax wage $(1 - \tau_t)\bar{l}W_t$; agents are unemployed when $\epsilon = 0$, and earn the unemployment benefit μW_t . Here W_t is the wage, τ_t is the tax rate, and μ is a model parameter denoting the fraction of wage.

¹As discussed in Den Haan, Judd and Juillard (2010), the structure is closely aligned with that of Krusell and Smith (1998) except for the introduction of income taxation and an unemployment benefit, which alone make the agents’ budget constraint occasionally binding.

The market is incomplete, featuring the capital borrowing constraint $k_t \geq 0$; the net rate of return on capital is $R_t - \delta$, where R_t is the market-determined interest rate and δ is the fixed depreciation rate. The maximization problem of agent i is as follows:

$$\mathbb{E} \sum_{t=0}^{\infty} \beta^t \frac{(c_t)^{1-\gamma} - 1}{1-\gamma}$$

subject to

$$c_t + k_{t+1} = R_t k_t + [(1 - \tau_t) \bar{l} \epsilon_t + \mu (1 - \epsilon_t)] W_t + (1 - \delta) k_t$$

Here c_t is the consumption level chosen by agent i and k_t is the beginning-of-period capital. We can safely write the equality budget constraint due to the satisfaction of Inada condition of this problem.

Firms and the Government

The good market is competitive and the production follows a Cobb-Douglas form:

$$Y_t = Z_t K_t^\alpha (\bar{l} L_t)^{1-\alpha}$$

where K is the per capita capital, L is the employment rate, and $\alpha \in [0, 1]$ is the capital sharing. Z_t is a binary aggregate productivity shock: $Z_t \in \{Z_b, Z_g\}$.

The government keeps its budget balanced each period by redistributing all tax revenue from the employed to the unemployed.

The firms' first-order optimality together with the government's budget constraint determine the prices and tax rate

$$R_t = \alpha Z_t \left(\frac{K_t}{\bar{l} L_t} \right)^{\alpha-1}, \quad W_t = (1 - \alpha) Z_t \left(\frac{K_t}{\bar{l} L_t} \right)^{\alpha}, \quad \tau_t = \frac{\mu(1 - L_t)}{\bar{l} L_t} \quad (1)$$

Shocks

The aggregate shock Z_t follows a first-order Markov structure. The idiosyncratic shock ϵ_t is correlated to the aggregate shock by following a first-order Markov structure conditional on the transition of Z_t , and conforms to the law of large numbers.

To be more concrete, the transition of (ϵ_t, Z_t) is characterized by a 4×4 transition matrix $\mathbf{\Pi}$, whose element $\pi_{\epsilon\epsilon'ZZ'}$ denotes the probability of transitioning to (ϵ', Z') from the current state (ϵ, Z) . The calibration is such that the unemployment rate $u_t = 1 - L_t$ is a function of the aggregate shock: $u(Z_b) = u_b > u(Z_g) = u_g$.

The Recursive Equilibrium

To analyze the recursive equilibrium, agents' state variables for decision-making are divided into individual states (k, ϵ) and aggregate states $(Z, \mathbf{\Gamma})$; the latter are used for computing current prices and predicting future prices. In a rational expectations equilibrium, $\mathbf{\Gamma}$ represents the joint distribution of agents over capital holdings k and employment status ϵ . Since ϵ is binary, the joint distribution function $\mathbf{\Gamma}(k, \epsilon)$ comprises two one-dimensional continuous distribution functions on k : $\mathbf{\Gamma}(k, 0)$ and $\mathbf{\Gamma}(k, 1)$. For simplicity, I assume that the agents observe only the marginal distribution of k . This assumption facilitates focusing on the continuous distribution function of k without the need to additionally consider ϵ .²

As discussed, the motion of Z is characterized by the exogenous matrix $\mathbf{\Pi}$; the law of motion of $\mathbf{\Gamma}$ is denoted by $\mathbf{H} : \mathbf{\Gamma}' = \mathbf{H}(\mathbf{\Gamma}, Z, Z')$. The agents' problem can therefore be expressed recursively as

$$\mathbf{V}(k, \epsilon; Z, \mathbf{\Gamma}) = \max_{k'} \{ \mathbf{U}(c) + \beta \mathbb{E}[\mathbf{V}(k', \epsilon'; Z, \mathbf{\Gamma}') \mid \epsilon, Z] \} \quad (2)$$

subject to

$$c + k' = Rk + [(1 - \tau)\bar{l}\epsilon + \mu(1 - \epsilon)]W + (1 - \delta)k, \quad (3)$$

$$\epsilon', Z' \sim \mathbf{\Pi}(\epsilon, Z), \quad (4)$$

$$\mathbf{\Gamma}' = \mathbf{H}(\mathbf{\Gamma}, Z, Z'), \quad (5)$$

$$k' \geq 0 \quad (6)$$

For a particular value function $\mathbf{V}(k, \epsilon, Z, \mathbf{\Gamma})$, the corresponding policy function is denoted by $k' = \mathbf{g}(k, \epsilon, Z, \mathbf{\Gamma})$. The solutions to 2 subject to 3, 4, 5, 6 are denoted as $\mathbf{V}^*(\cdot)$ and $\mathbf{g}^*(\cdot)$.

A recursive competitive equilibrium is then defined by a law of motion $\mathbf{H}(\cdot)$, a pair of individual functions $\mathbf{V}^*(\cdot)$ and $\mathbf{g}^*(\cdot)$, and the price system (R, W, τ) such that (i) $\mathbf{V}^*(\cdot)$ and $\mathbf{g}^*(\cdot)$ solve the agents' optimality problem, (ii) R, W and τ are competitive and (iii) $\mathbf{H}(\cdot)$ is generated by $\mathbf{g}^*(\cdot)$.

The main computational challenge for this problem lies in the fact that the endogenous state variable $\mathbf{\Gamma}$ is a function, thus constituting an infinite-dimensional object. Additionally, the law of motion for the aggregates, $\mathbf{H}$, is a nontrivial function, compounded by the fact that the policy functions $\mathbf{g}$ are nonlinear and exhibit a kink due to the borrowing constraint.

²Section 6.2 relaxes the binary employment shock to a continuous idiosyncratic productivity, in which case the relevant object is the genuinely two-dimensional joint distribution $\mu(k, \epsilon)$ and the operator becomes a two-dimensional map; the framework carries this at the same accuracy.

3 Comparison of Computational Strategies

Below, I briefly discuss two primary frameworks from the literature that address the challenges mentioned in the introduction: the KS framework that characterizes the distribution using a select number of moments and tracks only the motion of these moments and the NN framework that represents the distribution through a finite (and potentially large) number of simulated agents, with aggregate motion being the cumulative transition of all simulated agents adhering to a specific policy. Subsequently, I introduce my computational strategy, the operator learning framework, proposed in this paper.

KS Framework

The KS framework proposes approximating $\mathbf{\Gamma}$ by a finite set of moments $\mathbf{m} = (m_1, \dots, m_I)$. The aggregate motion is then approximated by the motions of these moments: $\mathbf{H}_m : \mathbf{m}' = \mathbf{H}_m(\mathbf{m}, Z, Z')$. For instance, in [Krusell and Smith \(1998\)](#), I is chosen to be 1, so $\mathbf{m} = (K)$, meaning that only the mean of capital is used to characterize the distribution. In the economic model studied in this paper, denote the policy function implementing the KS framework as $\mathbf{g}_{\text{KS}}(k, \epsilon, Z, K)$, which is the solution to the following:

$$\mathbf{V}(k, \epsilon; Z, K) = \max_{k'} \{ \mathbf{U}(c) + \beta \mathbb{E} [\mathbf{V}(k', \epsilon'; Z', K') \mid \epsilon, Z] \} \quad (7)$$

subject to

$$\begin{aligned} \log K' &= a_0 + a_1 \log K, & \text{if } Z = Z_b, \\ \log K' &= b_0 + b_1 \log K, & \text{if } Z = Z_g \end{aligned} \quad (8)$$

along with the budget constraint [\(3\)](#), the transitions of exogenous shocks [\(4\)](#), and the borrowing constraint [\(6\)](#). The KS framework implements a nested fixed-point optimization scheme that alternates between solving the agents' problem given the belief about the law of motion of moments and updating the belief given the agents' policy.

Conceptually, the KS algorithm approximates a computationally intractable infinite-dimensional problem by a computationally tractable low-dimensional problem. It is tractable, intuitive, and fast. However, it has two main limitations: Theoretically, it implicitly imposes the bounded rationality assumption that agents observe only the set of moments and perceive only the motion of those moments. Practically, the moment selection is arbitrary; the sufficiency for the first I moments to characterize the distribution is case-dependent, and if I is large enough, the dimensionality of the problem becomes too high, making the grid-based projection method computationally inefficient. The discussion is summarized in the [Table 2](#) below.

Pros	Cons
• Makes the problem tractable • Intuitive • Fast	• Assumes bounded rationality • Arbitrary moment selection

Table 2: Summary of KS Framework

NN Framework

The NN framework, instead, incorporates the full information of the distribution into the state variables using the individual states of all simulated agents. This transforms the infinite-dimensional original problem into a possibly high but finite-dimensional problem, where the dimensionality depends on the number of agents simulated. The aggregate law of motion is then approximated by the transitions of simulated agents based on a policy function. For the model in Section 2, denote the policy function implementing the NN framework as $\mathbf{g}_{\text{NN}}(k, \epsilon, Z, K)$, which is the solution to the following recursive form of the agents' problem:

$$\mathbf{V}(k, \epsilon; Z, \{k\}_{i=1}^N) = \max_{k'} \left\{ \mathbf{U}(c) + \beta \mathbb{E} [\mathbf{V}(k', \epsilon'; Z', \{k'\}_{i=1}^N) \mid Z] \right\} \quad (9)$$

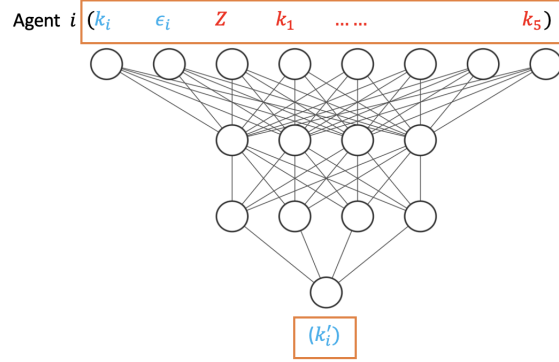
subject to

$$k'_i = \mathbf{g}(k_i, \epsilon_i, Z, \{k\}_{i=1}^N) \quad \text{for } i = 1, \dots, N \quad (10)$$

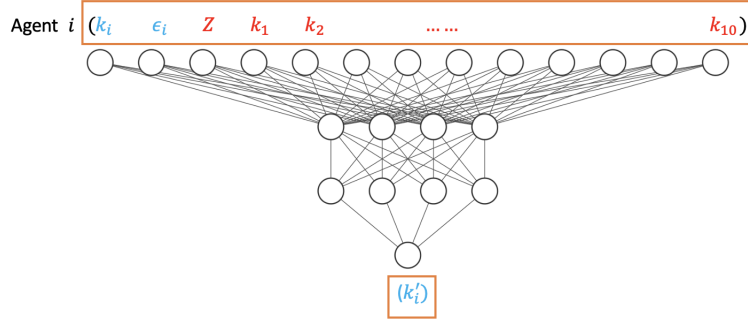
and the budget constraint (3), the transitions of exogenous shocks (4), and the borrowing constraint (6).

Since the number of agents is assumed to be large enough to approximate the continuous distribution function, to overcome the curse of dimensionality, the literature proposes using a feed-forward neural network as the function approximator for $\mathbf{g}_{\text{NN}}$. The feed-forward neural network has been shown to perform well in model reduction by condensing information and extracting features from the input to the hidden layers; see Goodfellow et al. (2016) for a review.

The NN framework addresses the issues in the KS framework, keeping the consequential high-dimensional problem tractable by adopting deep learning techniques. However, there are a few limitations that hinder computational efficiency. First, there is a trade-off in the choice of N : an increase in N leads to a better approximation of the continuous distribution function but also to a proportionally larger size of the network. More specifically, the size of the input layer grows linearly with N . If N is sufficiently large and the size of the hidden layers remains fixed, the network size grows roughly linearly with N . Figure 1 demonstrates this.



(a) 5 simulated agents



(b) 10 simulated agents

Figure 1: An illustration of the neural network size variation with the number of simulated agents. The above figure shows a network with 5 simulated agents, while the figure below depicts a larger network with 10 simulated agents, demonstrating the increase in the size of the input layer corresponding to the number of simulated agents, as discussed in the main text.

Moreover, in simulations where all agents transition to the next states based on the policy parameterized by the neural network (and in training, if all agents’ states are used as training data), the computational cost is quadratic in N since there are more agents (rows) and a higher-dimensional state (columns) for each agent. Figure 2 demonstrates this.

The scaling from the two aspects discussed above makes the computational costs heavily dependent on N . The second limitation is that the NN framework solution is tied to a chosen N ; the neural network architecture, especially the input layer dimension, is fixed once chosen. This inflexibility can cause issues when, for example, a larger number of simulated agents is required for evaluating the solution, or if the model setup includes overlapping-generation features so the population size may vary over time.

Finally, the discussed NN framework does not consider two features of the data structure: permutation-invariance and aggregation-sharing. Permutation-invariance means that the policy function $\mathbf{g}_{\text{NN}}$ should be invariant under permutations of the agents’ ordering. However, the plug-in vector $\{k\}_{i=1}^N$ does not encode this information, so it would take a significant amount of computational time for the neural network to learn this pattern. Aggregation-sharing means that all simulated agents share the same economic environment, and they have a large portion of state variables the same as each other. However, the NN framework implicitly considers each agent independent when processing her state vector. The discussion is summarized in the Table 3 below.

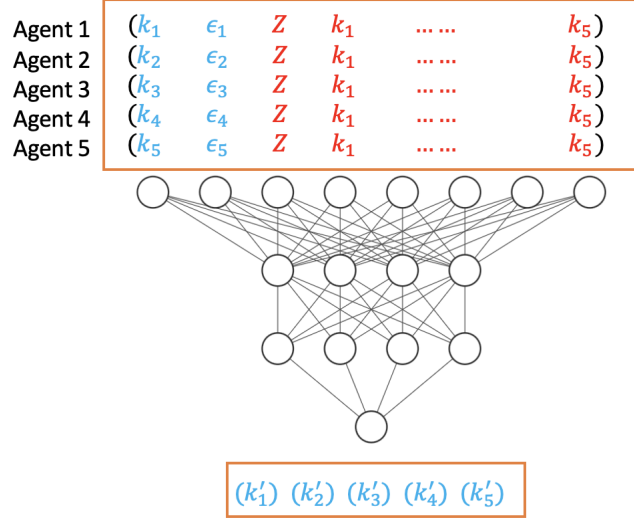
Pros	Cons
• Addresses the issues in the KS	• Computational costs heavily depend on N • Solution is fixed to a given N • Does not use data structure features like permutation-invariance and aggregation-sharing

Table 3: Summary of the NN Framework

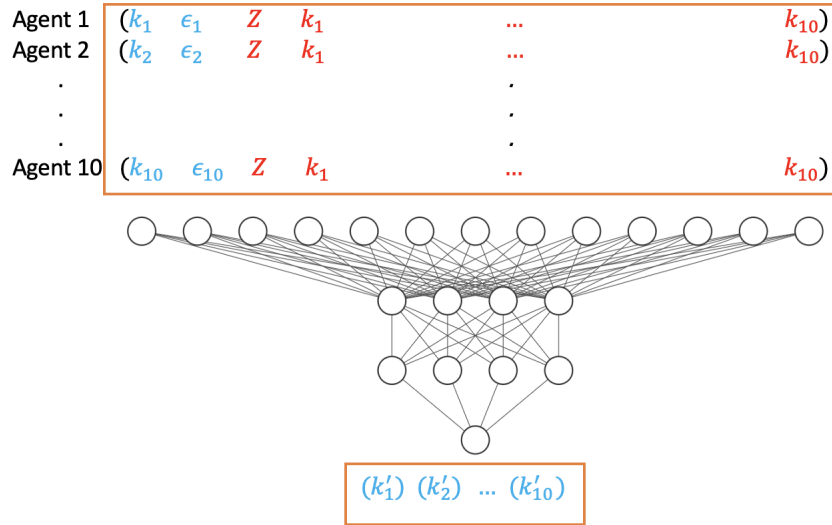
This Paper: Operator Learning Framework

The two frameworks discussed so far share a common intuition: they approximate the distribution function with a “distribution vector”. This can be achieved either parsimoniously in parametric form (KS) or in a non-parametric form with sufficient information (NN), aiming to reduce the policy function back to a case where the arguments include only a finite-dimensional vector.

This paper proposes an alternative view by reformulating the policy function into an operator form:



(a) With 5 simulated agents.



(b) With 10 simulated agents.

Figure 2: Illustration of the quadratic computational cost in neural network simulations. The subfigure above demonstrates the network with 5 simulated agents, and the subfigure below with 10 simulated agents, emphasizing the increased computational burden as the number of agents, and thus the dimensionality, grows.

$$\mathbf{g}(k, \epsilon, Z, \mathbf{\Gamma}) := \mathbf{G}(\mathbf{\Gamma})(k, \epsilon, Z) \quad (11)$$

Here, processing the state $(k, \epsilon, Z, \mathbf{\Gamma})$ into the policy function $\mathbf{g}$ is decomposed into two steps: firstly, an operator $\mathbf{G}$ takes the function $\mathbf{\Gamma}$ as its sole input and outputs another function $\mathbf{G}(\mathbf{\Gamma})$, which we call the “conditional policy function” henceforth; secondly, the vectors (k, ϵ, Z) serve as inputs to the function $\mathbf{G}(\mathbf{\Gamma})$.

While the input and output functions of the operator $\mathbf{G}$ do not necessarily need to be defined on the same domain, for simplicity, in the case studied, this alignment is effortlessly achieved. Given that both ϵ and Z are binary, $\mathbf{G}(\mathbf{\Gamma})(k, \epsilon, Z)$ can be interpreted as four separate 1-dimensional continuous functions of k , each defined on the same domain as the distribution function $\mathbf{\Gamma}(k)$.³

With this formulation, finding the solution for the policy function $\mathbf{g}$ translates to finding the solution for the operator $\mathbf{G}$ that satisfies Equation (11) for all states $(k, \epsilon, Z, \mathbf{\Gamma})$.

The question then arises: what are the benefits of this operator formulation? First, consider the parameterization of $\mathbf{G}$: if $\mathbf{G}$ is parameterized by a feed-forward neural network (or another functional form), there would be no distinction from the existing framework. To explore an alternative parameterization, we note the role of the distribution function $\mathbf{\Gamma}$ in agents’ decision-making: it integrates from the individual level to the aggregate level. Ideally, the operator $\mathbf{G}$ should involve an integral operation to transform the input distribution function into the output conditional policy function.

This paper adopts a machine-learning tool known as the neural operator to parameterize $\mathbf{G}$, a generalization of neural networks that maps between infinite-dimensional function spaces. Intuitively, the neural operator is composed of linear integral operators and nonlinear activation functions. It has a critical property, discretization invariance, meaning that the size of the neural operator remains constant regardless of the discretization of the input and output functions. This property addresses the first two disadvantages discussed in the NN framework context. Section 4 provides an introduction.

Furthermore, the operator formulation inherently includes the properties of permutation-invariance and aggregation-sharing. To illustrate, consider $\mathbf{\Gamma}$ explicitly as a cumulative distribution function (CDF), with an empirical representation consisting of two vectors: the input vector $\{\bar{k}_1, \dots, \bar{k}_J\}$ in ascending order, where $k_j \in [k_{\min}, k_{\max}]$ for $j = 1, \dots, J$, and the output vector $\{\frac{1}{J}, \dots, \frac{J}{J}\}$. This vector is constructed from the cross-sectional simulated agents $\{k_1, \dots, k_N\}$ with $J \leq N$. The transition from the non-ordered set of simulated agents $\{k_1, \dots, k_N\}$ to the ordered vector $\{\bar{k}_1, \dots, \bar{k}_J\}$ implicitly utilizes permutation-invariance.

The operator formulation’s two-step processing of state variables directly exhibits aggregation-sharing: since only the distribution function is processed in the first step, which is shared

³In other cases, e.g., when both ϵ and Z are continuous, we can consider $\mathbf{\Gamma}(k) := \mathbf{\Gamma}(k, \epsilon, Z)$ as a 3-dimensional continuous function, thereby defining $\mathbf{\Gamma}$ and $\mathbf{G}(\mathbf{\Gamma})$ on the same domain once again.

by all simulated agents, this processing is done only once for all agents, rather than being redundantly repeated for each one. Figure 3 provides a clear illustration.

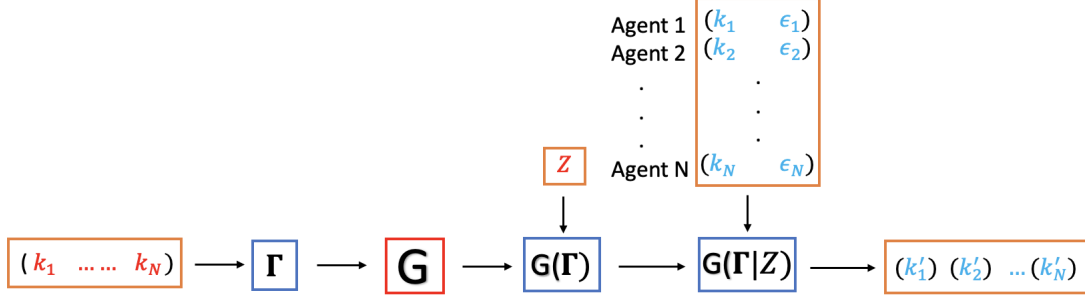


Figure 3: The processing flow of the neural operator, illustrating the transformation from the distribution function to the conditional policy function. This figure also demonstrates the permutation-invariance and aggregation-sharing properties discussed in the main text.

To conclude this section, the operator formulation’s defining feature is that it conditions on Γ as a function, committing to no finite, modeler-chosen representation in advance; discretization invariance, permutation invariance, and aggregation sharing follow as *consequences* of that choice rather than being the point itself. Whether this bottleneck-free conditioning translates into an accuracy or efficiency advantage over the KS and NN benchmarks, and where it does not, is evaluated across four economies in Section 6.

4 Neural Operator

This section introduces the neural operator, a concept recently proposed in the machine learning literature (Li et al., 2020c,a,b). The objective is to approximate a mapping between function spaces. In this discussion, I use notions that match the studied economic model, where functions have a 1-dimensional input $k \in [k_{\min}, k_{\max}]$ and produce a 1-dimensional vector output; for a comprehensive discussion, see Kovachki et al. (2021).

Consider a set of distribution functions $\mathcal{T} : \{\Gamma \in \mathcal{T}, \Gamma : [k_{\min}, k_{\max}] \rightarrow \mathbb{R}\}$, a set of conditional policy functions $\bar{\mathcal{G}}_{\Gamma} : \{\bar{\mathbf{g}}_{\Gamma} \in \bar{\mathcal{G}}_{\Gamma}, \bar{\mathbf{g}}_{\Gamma} : [k_{\min}, k_{\max}] \rightarrow [k_{\min}, k_{\max}]\}$, and a mapping $\mathbf{G} : \mathcal{T} \rightarrow \bar{\mathcal{G}}_{\Gamma}$ such that $\bar{\mathbf{g}}_{\Gamma}(k) = \mathbf{G}(\Gamma)(k)$. Our aim is to parameterize $\mathbf{G}$ with $\mathbf{G}_{\theta}$ to satisfy:

$$\mathbf{G}_{\theta}(\Gamma)(k) \approx \mathbf{G}(\Gamma)(k), \quad \text{for } \Gamma \in \mathcal{T} \quad \text{and} \quad k \in [k_{\min}, k_{\max}], \quad (12)$$

according to a particular measure.

A neural operator with L layers is a sequence of operations in function space, described

by:

$$\bar{\mathbf{g}}_{\mathbf{r}}(k) := \mathbf{G}_{\theta}(\mathbf{\Gamma})(k) = (\mathcal{Q} \circ \sigma_L \circ \Phi_L \circ \dots \circ \sigma_1 \circ \Phi_1 \circ \mathcal{P}(\mathbf{\Gamma}))(k). \quad (13)$$

In layer ℓ , $v_{\ell+1}(k) = \sigma_{\ell} \circ \Phi_{\ell}(v_{\ell})(k)$. Here,

$$(\Phi_{\ell} v_{\ell})(k) = \int_{q \in Q} \phi_{\ell}(k, q) v_{\ell}(q) dq + (W_{\ell} v_{\ell})(k), \quad (14)$$

is an integral kernel operation plus a linear transformation W_{ℓ} . The function σ_{ℓ} is an element-wise nonlinear transformation, e.g., $\text{ReLU}(x) = \max(x, 0)$ and $\text{Sigmoid}(x) = \frac{1}{1 + \exp(-x)}$. The mappings $\mathcal{P} : \mathbb{R} \rightarrow \mathbb{R}^{v_0}$ and $\mathcal{Q} : \mathbb{R}^{v_L} \rightarrow \mathbb{R}$ are element-wise lifting and projection mappings, respectively.

In practice, both the input function $\mathbf{\Gamma}(k)$ and the output function $\mathbf{G}(\mathbf{\Gamma})(k)$ are discretely represented at sufficiently many but finite locations $\{\bar{k}_1, \dots, \bar{k}_J\} \in [k_{\min}, k_{\max}]$, referred to as "sensors" in the machine learning literature. The theoretical foundation of the neural operator's approximation capabilities is underpinned by the universal approximation theorem for operators, dating back to the seminal work by [Chen and Chen \(1995\)](#). I adopt a version of the theorem as presented by [Lu, Jin and Karniadakis \(2019\)](#):

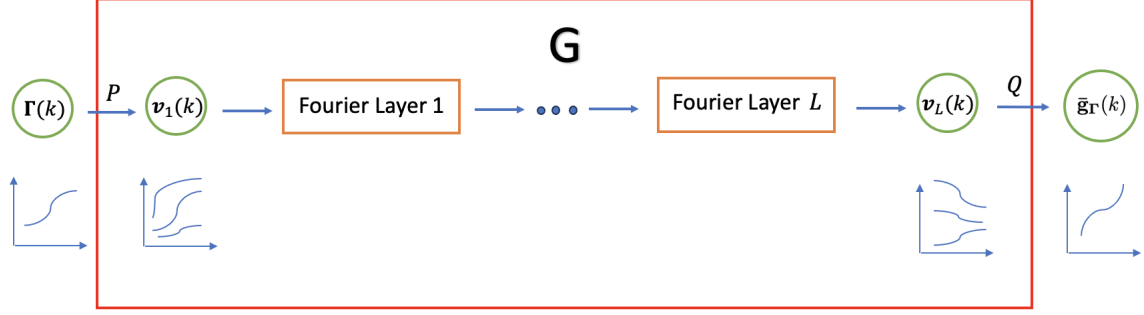
Theorem 1 (Universal Approximation Theorem for Operators ([Lu, Jin and Karniadakis, 2019](#))). *Let σ be a continuous non-polynomial activation function, and let X be a Banach space. Let $K_1 \subset X$ and $K_2 \subset \mathbb{R}^d$ be compact subsets of X and $\mathbb{R}^d$, respectively. Suppose V is a compact subset of $C(K_1)$, and G is a nonlinear continuous operator mapping V into $C(K_2)$. Then for any $\epsilon > 0$, there exist positive integers n, p, m , constants $c_i^p, \xi_{ij}^p, \theta_i^p, \zeta_p \in \mathbb{R}$, vectors $w_p \in \mathbb{R}^d$, and points $x_j \in K_1$ for $i = 1, \dots, I$, $p = 1, \dots, P$, $j = 1, \dots, J$, such that*

$$\left| \mathbf{G}(\mathbf{\Gamma})(y) - \sum_{p=1}^P \left(\sum_{i=1}^I c_i^p \sigma \left(\sum_{j=1}^J \xi_{ij}^p \mathbf{\Gamma}(k_j) + \theta_i^p \right) \right) \sigma(w_p \cdot y + \zeta_p) \right| < \epsilon,$$

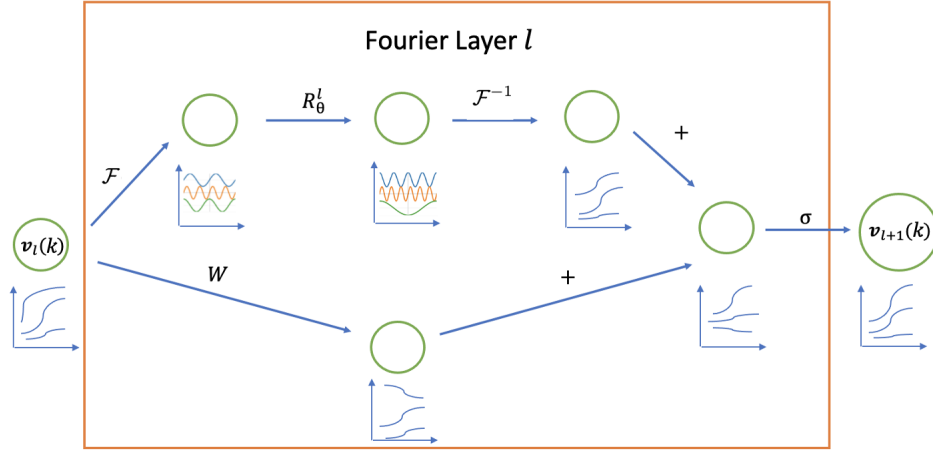
for all $\mathbf{\Gamma} \in V$ and $y \in K_2$.

Here the integral operator $\phi_{\ell}(k, q)$ is what aimed to learn from data. Various methods have been proposed in the literature to parameterize the integral operator $\phi_{\ell}(k, q)$ ([Li et al., 2020c,a](#)). In this paper, I adopt the Fourier Neural Operator (FNO) framework ([Li et al., 2020b](#)), which parameterizes the kernel ϕ_{ℓ} in the Fourier domain rather than the spatial domain and uses the Fast Fourier Transform (FFT) to compute Equation 14. Specifically, let $\phi_{\ell}(k, q) = \phi_{\ell}(k - q)$, signifying a convolution, thereby enabling the use of the Convolution Theorem:

Theorem 2 (Convolution Theorem). *Let f and g be functions with Fourier transforms $\mathcal{F}\{f\}$ and $\mathcal{F}\{g\}$, respectively. The Fourier transform of the convolution $f * g$ is the point-*



(a) The overall architecture of the Fourier Neural Operator.



(b) A detailed view of a single Fourier layer within the architecture.

Figure 4: Illustration of the Fourier Neural Operator (FNO) architecture. Subfigure (a) shows the overall architecture, and Subfigure (b) provides a detailed view of the components within a single Fourier layer.

wise product of $\mathcal{F}\{f\}$ and $\mathcal{F}\{g\}$. Mathematically, this relationship is expressed as:

$$\mathcal{F}\{f * g\}(k) = \mathcal{F}\{f\}(k) \cdot \mathcal{F}\{g\}(k).$$

Denote $\mathcal{F}$ and $\mathcal{F}^{-1}$ as the Fourier transform and inverse Fourier transform, respectively. Li et al. (2020b) suggest parameterizing $\mathcal{F}(\phi_\ell)$ with a function R_θ^ℓ in the Fourier domain, parameterized by θ :

$$\int_{q \in Q} \phi_\ell(k, q) v_\ell(q) dq = \mathcal{F}^{-1}(\mathcal{F}(\phi_\ell) \cdot (\mathcal{F}(v_\ell)))(k) \approx \mathcal{F}^{-1}(R_\theta^\ell \cdot (\mathcal{F}(v_\ell)))(k).$$

Figure 4 illustrates the architecture of the neural operator with each hidden layer represented as a Fourier layer.

Since the kernel function ϕ_ℓ is implicitly assumed to be periodic when it is transformed into the Fourier domain, it admits a Fourier series expansion and the frequency modes κ are discrete. In practice, the Fourier series is truncated at a maximal number of modes $\kappa_{\max}$ for a finite-dimensional parameterization. Therefore, R_θ^ℓ is a $(\kappa_{\max} \times d_{v_\ell} \times d_{v_{\ell+1}})$ complex-valued tensor. For computational complexity, the bulk of the computational cost lies in computing the Fourier transform $\mathcal{F}(v_\ell)$ and its inverse. General Fourier transforms have a complexity of $\mathcal{O}(J^2)$; however, since the Fourier series is truncated, the complexity reduces to $\mathcal{O}(J\kappa_{\max})$. The FFT has a complexity of $\mathcal{O}(J \log J)$, but it requires a uniform discretization of the function.

5 Implementation Details

Before turning to the implementation details, I restate the economic problem I intend to address, with a slight reformulation, as follows:

$$\mathbf{V}(k, \epsilon; Z, \mathbf{\Gamma}) = \max_{c, k'} \{ \mathbf{U}(c) + \beta \mathbb{E}[\mathbf{V}(k', \epsilon'; Z, \mathbf{\Gamma}') \mid \epsilon, Z] \}$$

subject to

$$\begin{aligned} m &= \mathbf{M}(k, \epsilon, Z, \mathbf{\Gamma}) \\ c &= \mathbf{C}(k, \epsilon, Z, \mathbf{\Gamma}) \\ k' &= m - c \geq 0 \\ \epsilon', Z' &\sim \mathbf{\Pi}(\epsilon, Z) \\ \mathbf{\Gamma}' &= \mathbf{H}(\mathbf{\Gamma}, Z, Z') \end{aligned}$$

and the solution is a policy function $k' = \mathbf{g}(k, \epsilon; Z, \mathbf{\Gamma})$ such that

$$\mathbf{g}(k, \epsilon; Z, \mathbf{\Gamma}) = \arg \max_{k'} \{ \mathbf{U}(c) + \beta \mathbb{E}[\mathbf{V}(k', \epsilon'; Z, \mathbf{\Gamma}') \mid \epsilon, Z] \} \quad (15)$$

The modification here is to represent the computation of the prices (R, W, τ) and, consequently, the agents' wealth m , as a function denoted by $\mathbf{M}$. The agents' policy is expressed in terms of the consumption policy $\mathbf{C}$. The capital in next period k' is then determined as the wealth minus consumption.

Approximation

Suppose there is a set of J grid points $\bar{\mathbf{k}} = (\bar{k}_1, \dots, \bar{k}_J) \in [0, k_{max}]$, which is called "sensors" in the literature. Also there is cumulative-distribution function (CDF) of $k \in [0, k_{max}]$ denoted as $\tilde{\mathbf{F}}$. The construction of $\bar{\mathbf{k}}$ and $\tilde{\mathbf{F}}$ are discussed soon. I use the upper bar symbol to denote those variables that are fixed during the training and the upper tilde symbol for those that are amendable. $\mathbf{\Gamma}$ can be represented as a $2 \times J$ vectors $\hat{\mathbf{\Gamma}} \equiv (\bar{\mathbf{k}}, \tilde{\mathbf{F}}(\bar{\mathbf{k}}))$.

Define an alternative policy function as the fraction of wealth to consume, and rewrite it into an operator form:

$$\zeta = \frac{c}{m} = \frac{\mathbf{C}(k, \epsilon, Z, \hat{\mathbf{\Gamma}})}{\mathbf{M}(k, \epsilon, Z, \hat{\mathbf{\Gamma}})} \equiv \mathbf{g}(k, \epsilon, Z, \hat{\mathbf{\Gamma}}) = \mathbf{G}(\hat{\mathbf{\Gamma}})(k, \epsilon, Z) \quad (16)$$

so that $k' = (1 - \zeta)m$. I use the neural operator $\mathbf{G}_\theta$, whose parameters are denoted θ , to approximate the operator $\mathbf{G}$:

$$\mathbf{g}_\theta(k, \epsilon, Z, \hat{\mathbf{\Gamma}}) \equiv \mathbf{G}_\theta(\hat{\mathbf{\Gamma}})(k, \epsilon, Z) \approx \mathbf{G}(\hat{\mathbf{\Gamma}})(k, \epsilon, Z)$$

That is, the neural operator $\mathbf{G}_\theta$ takes as input the $2 \times J$ vectors $\hat{\mathbf{\Gamma}}$ and output the $4 \times J$ vectors

$$\zeta_J \equiv \begin{bmatrix} (\hat{\zeta}_j, \dots, \hat{\zeta}_J)_{\epsilon=0, Z=Z_b} \\ (\hat{\zeta}_j, \dots, \hat{\zeta}_J)_{\epsilon=1, Z=Z_b} \\ (\hat{\zeta}_j, \dots, \hat{\zeta}_J)_{\epsilon=0, Z=Z_g} \\ (\hat{\zeta}_j, \dots, \hat{\zeta}_J)_{\epsilon=1, Z=Z_g} \end{bmatrix}$$

The Objective Function

Since the outputs of true $\mathbf{G}^*$ are unobserved, the training of neural operator cannot be formulated as a supervised learning task with a norm of the difference $\|\mathbf{G}_\theta(\hat{\mathbf{\Gamma}})(k, \epsilon, Z) - \mathbf{G}^*(\hat{\mathbf{\Gamma}})(k, \epsilon, Z)\|$. To construct an objective function, recall that the unique solution that solves the Bellman equation must satisfy the derived Euler equation in the absence of borrowing constraint:

$$\frac{du}{dc}(c) = \beta \mathbb{E}[(1 - \delta + R') \frac{du}{dc}(c')] \quad (17)$$

A standard approach in the literature is to define the relative Euler residual in, e.g., L_2 norm. For a given state $\omega \equiv (k, \epsilon, Z, \hat{\mathbf{\Gamma}})$ and a neural operator derived policy $\zeta = \mathbf{g}_\theta(\omega)$, define the unit-free Lagrange multiplier

$$h \equiv 1 - \frac{\beta \mathbb{E}[(1 - \delta + R') \frac{du}{dc}(\zeta' m')]}{\frac{du}{dc}(\zeta m)} \quad (18)$$

with $m = \mathbf{M}(\omega)$, $m' = \mathbf{M}(\omega')$, $\zeta' = \mathbf{g}_\theta(\omega')$ and R' the function of ω' according to 1. Then agents' optimality can be expressed in terms of the Kuhn-Tucker conditions:

$$h \geq 0, \quad k' \geq 0, \quad h k' = 0 \quad (19)$$

To get rid of the inequalities in 19 and construct an differentiable objective function, I rephrase the Kuhn-Tucker conditions in terms of the Fischer-Burmeister (FB) transformation with equality,

$$\Psi^{FB}(a, b) = a + b - \sqrt{a^2 + b^2} = 0 \quad (20)$$

with $a = 1 - \zeta$ and $b = h$. In this case the objective function is

$$\xi(\omega, \theta) \equiv \|\Psi^{FB}(1 - \zeta, h)\|^2 \quad (21)$$

Regarding the transition from ω to ω' : The expectation operator $\mathbb{E}$ is taken over all possible (ϵ', Z') according to the transition matrix $\mathbf{\Pi}$. $k' = (1 - \zeta)m$. $\mathbf{\Gamma}' \equiv (\bar{\mathbf{k}}, \tilde{\mathbf{F}}'(\bar{\mathbf{k}}))$ with the $\tilde{\mathbf{F}}'$ the updated empirical CDF while keeping the grids $\bar{\mathbf{k}}$ fixed.

In iteration ℓ , suppose there is a set of observations $\{\omega : \omega \in \Omega^\ell\}$, then the objective function is

$$\Xi^\ell(\theta) = \frac{1}{|\Omega^\ell|} \sum_{\omega \in \Omega^\ell} \xi(\omega, \theta) \quad (22)$$

and the parameters in the neural operator are updated used the gradient descent method:

$$\theta^{\ell+1} \leftarrow \theta^\ell - \lambda^\ell \nabla_\theta \Xi(\theta^\ell) \quad (23)$$

What remains is the construction of $\bar{\mathbf{k}}, \tilde{\mathbf{F}}$ and Ω^ℓ .

Data Generating Process

Denote $\{\mathcal{T} : \mathbf{\Gamma} \in \mathcal{T}\}$ the set of all possible $\mathbf{\Gamma}$. A natural candidate for Ω_l is the set $\mathbf{\Omega} \equiv [0, k_{max}] \times \{0, 1\} \times \{Z_b, Z_g\} \times \mathcal{T}$. However, while grids can be taken on $[0, k_{max}] \times \{0, 1\} \times \{Z_b, Z_g\}$, it is not possible on the $\mathcal{T}$. Therefore I adopt stochastic simulation and concentrate only on those $\mathbf{\Gamma}$ in the ergodic set $\hat{\mathbf{\Gamma}}_{\tilde{g}}$ (Judd et al., 2011), given a policy $\tilde{g}$.

To be concrete, a cross section of N agents is simulated. In each period, the agents are characterized by the cross sectional capital $\tilde{\mathbf{k}} = (\tilde{k}^1, \dots, \tilde{k}^N)$ in ascending order, the corresponding idiosyncratic risk $\tilde{\epsilon} = (\tilde{\epsilon}^1, \dots, \tilde{\epsilon}^N)$, the aggregate shock $\tilde{Z}$, and a shared policy for transition $\tilde{\mathbf{g}}$.

The empirical CDF $\tilde{\mathbf{F}}(k)$ is the interpolation of mapping $\tilde{\mathbf{k}}$ to $(\frac{1}{N}, \dots, \frac{N}{N})$:

$$\tilde{\mathbf{F}}(k) = \begin{cases} 0 & \text{if } k < \tilde{k}^1 \\ \frac{i}{N} + \left(\frac{k - \tilde{k}^i}{\tilde{k}^{i+1} - \tilde{k}^i} \right) \left(\frac{i+1}{N} - \frac{i}{N} \right) & \text{if } \tilde{k}^i \leq k < \tilde{k}^{i+1}, i = 1, 2, \dots, N-1 \\ 1 & \text{if } k \geq \tilde{k}^N \end{cases} \quad (24)$$

In each iteration ℓ , I run the stochastic simulation for T periods, discarding the beginning T_d periods for avoiding the impact of initial distribution, and select the time frame every t_{skip} periods to reduce the correlation between samples, so to obtain a set of $B = (T - T_d)/t_{skip}$ distributions $\{\tilde{\mathbf{F}}_b(\tilde{\mathbf{k}})\}_{b=1}^B$.

Now turn to the discussion for the sampling of (k, ϵ, Z) . In principle we can use the simulated data $(k^i, \epsilon^i, Z)_{i=1, b=1}^{N, B}$, which is in line with the pure stochastic simulation strategy discussed in [Maliar et al. \(2021\)](#). Instead, I adopt the grids method with the rationale that the infinite-dimensional object $\mathbf{\Gamma}$ is already isolated using stochastic simulation, with only three dimensions remaining, two of them binary. In the absence of the concerns on the curse-of-dimensionality, the grids method gives higher accuracy comparing to the stochastic simulation method([Judd et al., 2011](#)).

For the discretization of $k \in [0, k_{max}]$, since the policy function is highly non-linear around the borrowing constraint due to the presence of a kink-point([Aiyagari, 1994](#)), I choose J grid points $\bar{\mathbf{k}} = (\bar{k}_1, \dots, \bar{k}_J)$, following the polynomial rule([Maliar and Maliar, 2014](#)):

$$k_j = \left(\frac{j}{J} \right)^d (k_{\max} - k_{\min}) + k_{\min} \quad \text{for } j = 1, \dots, J \quad (25)$$

which enable more grids at the low capital levels and help in accurate approximation.

Note that the choice of grids is the same for both the discretization of the agents' individual state and the censoring of the distribution function. The rationale is twofold. For implementation, the output function of the operator $\mathbf{G}_\theta$ is implicitly censored on the same grids $\bar{\mathbf{k}}$ as the input function, so no extra effort is needed to query for new points; from the perspective of the economic theory, the fraction of hand-to-mouth agents is significant in aggregation due to their non-linear behaviors, therefore it is reasonable to have more censoring grids in the lower capital level when approximating the distribution function.

Pre-Training

A poor initialization of the policy operator is detrimental to the training, since the data (the distribution) is endogenously generated following the policy. To facilitate the convergence, I pre-train the operator using the solution of the following problem,

$$\mathbf{V}(k, \epsilon) = \max_{c, k'} \{ \mathbf{U}(c) + \beta \mathbb{E}[\mathbf{V}(k', \epsilon') \mid \epsilon] \} \quad (26)$$

subject to

$$\begin{aligned} m &= \mathbf{M}(k, \epsilon) \\ c &= \mathbf{C}(k, \epsilon) \\ k' &= m - c \\ \epsilon' &\sim \mathbf{\Pi}(\epsilon) \\ k' &\geq 0 \end{aligned}$$

i.e. the version without aggregate shock. This model has a static aggregation, and hence all the prices are fixed and agents' state includes only the individual variables. Using policy function iteration this problem can be solved in seconds.

Denote the solution policy function as $\mathbf{g}_{static}$, that is, the economy is static in aggregate level. The pre-training is then simply a supervised learning task with the objective function

$$\xi_{SL}(\omega, \theta) = \left\| \frac{g_{\theta}(\omega)}{\mathbf{g}_{static}(\omega)} - 1 \right\|^2 \quad \text{for } \omega \sim \mathbf{g}_{static} \quad (27)$$

$$\Xi_{SL}^{\ell}(\theta) = \frac{1}{|\Omega^{\ell}|} \sum_{\omega \in \Omega^{\ell}} \xi_{SL}(\omega, \theta) \quad (28)$$

That is, the ergodic set is generated by $\mathbf{g}_{static}$.

The Algorithm 1 below is the pseudo-code that fully describes the neural operator training algorithm. Note that the formal training process closely resembles the pre-training process, with two exceptions: 1) the objective function for optimization, and 2) the policy followed by the agents in the simulation.

6 Results

This section reports the numerical results across four economies. The first two are validation exercises on standard Krusell–Smith-type models in which approximate aggregation holds: the canonical economy (Section 6.1) and its extension to a continuous idiosyncratic

Algorithm 1 Neural Operator Training Algorithm

Step 0: Initialization:

1. Fourier neural operator $\mathbf{G}_\theta$
2. Solution $\mathbf{g}_{\text{static}}$ of the static problem 26 such that $\zeta_{\text{static}} = \mathbf{g}_{\text{static}}(k, \epsilon)$ and $k'_{\text{static}} = \zeta_{\text{static}} m$
3. Set of N agents with: aggregate shock sequence $(Z_0, \dots, Z_{T-1})$, panel of idiosyncratic shocks $(\epsilon_0^1, \dots, \epsilon_0^N), \dots, (\epsilon_{T-1}^1, \dots, \epsilon_{T-1}^N)$, initial cross sectional capital holding $(\tilde{k}_0^1, \dots, \tilde{k}_0^N)$
4. Set of J grids $\bar{\mathbf{k}} = (\bar{k}_1, \dots, \bar{k}_J)$ as per the polynomial rule 25

Step 1: Pre-training:

for ℓ from 1 to L_{pre} do

1. Sampling the Distributions:

for t from 0 to $T - 1$ do

1-1. Construct $\tilde{\mathbf{F}}_t$ using $(\tilde{k}_t^1, \dots, \tilde{k}_t^N)$ as per 24 and $\tilde{\mathbf{F}}_t \equiv (\bar{\mathbf{k}}, \tilde{\mathbf{F}}_t(\bar{\mathbf{k}}))$

1-2. Compute cross sectional wealth $(\tilde{m}_t^1, \dots, \tilde{m}_t^N)$ as per the model specification $m = \mathbf{M}(k, \epsilon, Z, \hat{\mathbf{F}})$

1-3. Agents transit from $(k_t^1, \dots, k_t^N)$ to $(k_{t+1}^1, \dots, k_{t+1}^N)$ using $\mathbf{g}_{\text{static}}$

end for

2. Data Processing:

2-1. Discard initial T_d periods; select every t_{skip} to obtain $\{\tilde{\mathbf{F}}_b, Z_b\}_{b=1}^B$

2-2. Set initial capitals for next iteration: $(\tilde{k}_0^1, \dots, \tilde{k}_0^N) = (k_T^1, \dots, k_T^N)$

3. Parameters Update:

3-1 Set $\mathbf{G}_\theta$ to the train mode

for b from 1 to B do

3-2 construct the data set $\{\omega : \omega \in \Omega_\ell\}$, where $\Omega_\ell = \bar{\mathbf{k}} \times \{0, 1\} \times \{Z_b\} \times \{\tilde{\mathbf{F}}_b\}$

3-3. Update using objective functions 27 and 28

end for

end for

Step 2: Training:

for ℓ from 1 to L_{train} do

1. Sampling the Distributions:

Set $\mathbf{G}_\theta$ to the evaluation mode

for t from 0 to $T - 1$ do

Same as in Step 1 but agents transit using g_{θ^ℓ}

end for

2. Data Processing:

Same as in Step 1

3. Parameters Update:

Set $\mathbf{G}_\theta$ to the train mode

for b from 1 to B do

Same as in Step 1 but update using objective functions 21 and 22

end for

end for

shock (Section 6.2). In both, the shape of the cross-sectional distribution is nearly invariant along the ergodic path, so that only its mean tracks the aggregate cycle. A well-specified moment method therefore suffices, and the operator’s role is to recover that solution accurately and to keep the simulated economy stable, not to exploit distributional shape. The last two economies are those in which the operator’s distinctive value appears, because the shape of the distribution varies along the ergodic path and approximate aggregation fails: a two-asset HANK economy in which the liquid return depends on the joint shape of wealth (Section 6.3), and a decreasing-returns firm economy in which the wage depends on the dispersion of the firm distribution (Section 6.4). The distinction is sharper than in classic Krusell–Smith: in the canonical economies current prices are functions of the mean by construction, and approximate aggregation concerns only the forecast rule for aggregate capital, whereas in the last two the equilibrium price is itself a nonlinear functional of the distribution’s shape. Section 6.5 summarizes. All experiments use double-precision arithmetic and fixed random seeds and are reproducible from the replication code. Table 4 previews the four economies and the headline finding from each.

Table 4: The four economies and what each tests.

Economy	Heterogeneity / friction	Distribution shape	Headline finding
Canonical KS	employment shock, aggregate TFP	$\approx$ static (approx. holds)	Operator solves to $< 1\%$ Euler in minutes and simulates on a consistent ergodic path.
Continuous- ϵ KS	continuous idiosyncratic productivity	$\approx$ static (holds)	The reformulation scales to a genuinely 2-D joint distribution at comparable accuracy.
Two-asset HANK	liquid/illiquid, Calvo friction	varies (fails; price R^b)	Conditioning on the full $\mathbf{\Gamma}$ recovers the shape-dependent price; moment truncation and a moment-bottleneck network do not.
DRS firms	idiosyncratic productivity, dispersion-dependent wage	varies (fails; Jensen)	Operator matches a rich, well-specified moment solution without hand-picking moments; naive moments miss the wage.

6.1 The canonical Krusell–Smith economy

We first validate the framework on the economy of Section 2. We simulate a cross-section of $N = 1,000$ agents, the largest used by Maliar et al. (2021), and discretize each distribution channel on $J = 100$ grid points. The operator uses 64 retained Fourier modes and 16 hidden channels, giving 71,188 real parameters. It is pre-trained against the static (no-aggregate-shock) policy for under a minute and then trained on the Euler residual along the simulated ergodic path, with cosine learning-rate decay and gradient clipping.

The operator solves the model accurately. The Euler-residual training loss falls to $\sim 10^{-5}$ within roughly five minutes. To assess robustness rather than a single run, we repeat the entire procedure over ten random seeds, varying both the network initialization and the simulated shock path; Figure 5a plots the loss across seeds (median with a 10–90% band). On a fresh out-of-sample shock path the operator attains a median relative Euler error of 0.29% (interquartile range [0.18%, 0.34%]), below the one-percent threshold that Maliar et al. (2021) emphasize as demanding for a policy with a borrowing-constraint kink. Figure 5b displays a representative input distribution and the recovered conditional policy $\mathbf{g}(k, \epsilon, Z; \mathbf{\Gamma})$ for the four (ϵ, Z) combinations. The low-capital zoom makes the kink explicit: unemployed agents are pinned at the borrowing constraint up to a threshold and then lift off. This is direct evidence of a correctly solved constrained policy, and the moment-based KS reference nearly overlays the operator solution.

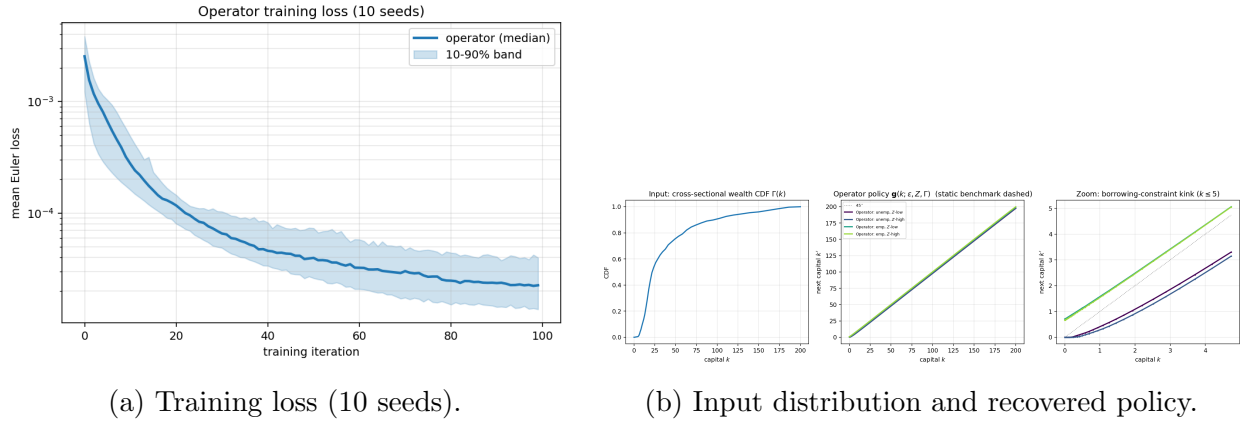


Figure 5: Canonical KS economy. Left: Euler training loss against iteration across ten seeds (median with 10–90% band), falling to $\sim 10^{-5}$. Right: a representative input distribution $\Gamma(k)$ and the recovered operator policy $\mathbf{g}(k, \epsilon, Z; \mathbf{\Gamma})$ for the four (ϵ, Z) cases, with a low-capital zoom exposing the borrowing-constraint kink.

The solved economy is stable, and a matched network is not. Simulated forward under its own policy, the operator’s economy settles on a consistent ergodic path at $K \approx 41$, matching the moment-based KS solution (den-Haan forecast error 0.71%). This stability is

not automatic. We benchmark the operator against the deep network of [Maliar et al. \(2021\)](#), which ingests the full N -agent cross-section as a 1,003-dimensional input (68,481 parameters, matched to the operator’s 71,188), under identical pre-training, Euler-residual training, and budget over the same ten seeds (Table 5). The operator is roughly an order of magnitude more accurate on the out-of-sample Euler metric (median 0.29% against 6.6%), but the larger difference is one of generalization: the network’s simulated economy diverges in nine of ten seeds, its aggregate capital collapsing toward zero or exploding (range [2, 108]) even though its training loss is finite, whereas the operator lands within 20% of the true level in nine of ten. The network ingests the cross-section as a high-dimensional, unstructured input and overfits the training states, while the operator treats it as a smooth function; this identifies the operator’s structured representation, rather than distributional information per se, as the source of its stability. Because a moment method already succeeds on this economy, the comparison establishes the operator’s accuracy and stability rather than a distributional advantage over moments.

Table 5: Canonical KS economy: operator against the deep feed-forward benchmark of [Maliar et al. \(2021\)](#), over ten seeds.

Method	Params	Euler, med. [IQR]	Ergodic K	Diverged
Operator (FNO)	71,188	0.29% [0.18, 0.34]	≈ 41	1/10
Deep NN	68,481	6.59% [5.10, 9.83]	[2, 108]	9/10
Moment-KS	—	0.71%	≈ 40	—

Ten seeds, varying network initialization and the simulated shock path. Both learning methods use cosine LR decay and gradient clipping and the same pre-train and train budget. Euler error is the per-seed out-of-sample relative residual (median across seeds, interquartile range); the Moment-KS entry reports its den-Haan forecast error rather than an Euler residual. “Diverged” counts seeds whose simulated ergodic capital departs more than 20% from the true level ($K \approx 41$) or whose Euler residual is non-finite, a generalization metric on which neither learning method is trained.

6.2 A continuous idiosyncratic shock

The canonical economy has a binary employment shock, so the operator conditions on a one-dimensional marginal over capital. We now replace employment with a continuous idiosyncratic productivity following an AR(1) process, discretized by the Rouwenhorst method, so that the relevant cross-section is the genuinely two-dimensional joint distribution $\mu(k, \epsilon)$ and the operator becomes a two-dimensional Fourier neural operator. This tests whether the reformulation scales to a multi-dimensional individual state rather than exploiting the binary structure of the canonical model.

It does. Figure 6a shows the Euler residual falling to $\sim 10^{-3}$ while the ergodic aggregate capital settles at $K \approx 16.8$, above the no-aggregate-shock level $K^* \approx 14.0$; the ergodic mean

lies above the no-shock level, reflecting the economy’s response to aggregate risk. Over five seeds the out-of-sample Euler error has a median near 1%. Figure 6b shows the input as a genuine two-dimensional density over the (k, ϵ) plane and the recovered policy, with the low-capital zoom resolving the productivity spread (higher productivity, more saving). As in the canonical economy, approximate aggregation holds: the shape of μ is nearly invariant along the ergodic path, so this is again a validation exercise. The value is that the operator carries the two-dimensional distribution at comparable accuracy and cost, and does not rely on shape variation.

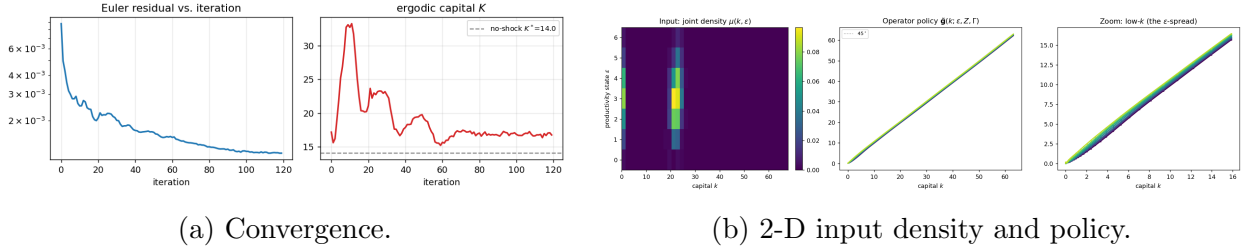


Figure 6: Continuous- ϵ economy. Left: Euler residual (to $\sim 10^{-3}$) and ergodic capital ($K \approx 16.8$, above the no-shock $K^* \approx 14.0$). Right: the input joint density $\mu(k, \epsilon)$ as a two-dimensional heatmap and the recovered policy per productivity state, with a low-capital zoom.

6.3 A two-asset economy where approximate aggregation fails

Our first demanding test is a discrete-time two-asset HANK economy in the spirit of [Kaplan et al. \(2018\)](#). Households hold illiquid capital and a liquid bond and may rebalance the illiquid account only occasionally, here through a Calvo (random) adjustment friction, a discrete-time simplification of the fixed adjustment cost in continuous-time two-asset models, and the liquid return R^b clears the bond market. We calibrate the economy to the regime in which R^b depends on the shape of the joint wealth distribution beyond its low-order moments, so that, unlike in Sections 6.1–6.2, approximate aggregation fails by construction and the shape of the distribution varies materially along the ergodic path.

The price is a functional of the joint shape. Figure 7 makes the failure direct. Holding the marginals fixed, we forecast the market-clearing R^b from the cross-sectional distribution two ways. A moment-based Krusell–Smith rule, a ridge regression on the low-order comoments, recovers it only diffusely ($R^2 \approx 0.5$), whereas conditioning on the full distribution recovers it almost exactly ($R^2 \approx 0.96$). The gap is the distributional information that low-order moments discard.

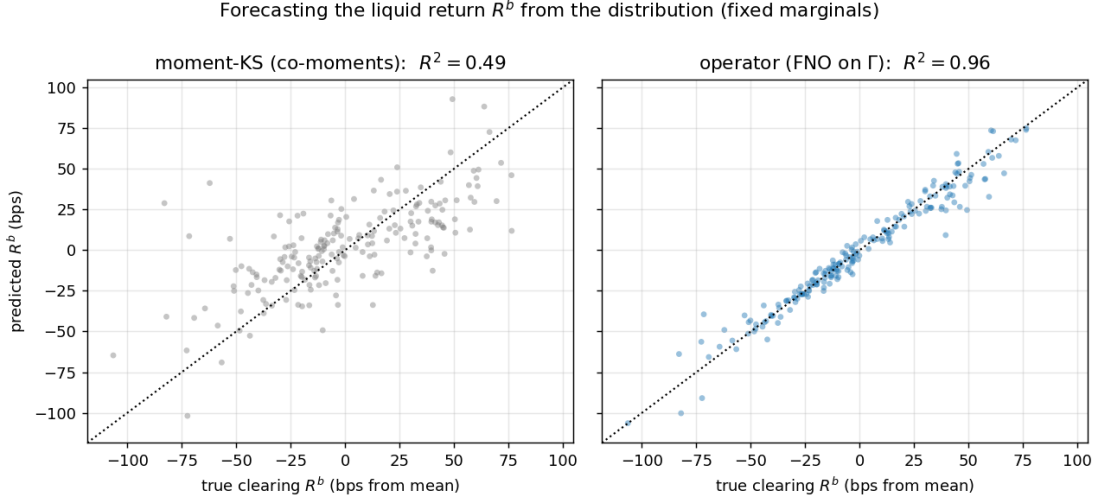


Figure 7: Two-asset HANK: forecasting the clearing liquid return R^b at fixed marginals. Moment-KS (a ridge on co-moments) leaves a diffuse cloud ($R^2 \approx 0.5$); conditioning on the full distribution recovers the price ($R^2 \approx 0.96$).

Solving and simulating. We solve the economy three ways, namely moment-based KS, the Euler-trained operator (a two-dimensional FNO on the joint distribution), and a flat multi-layer perceptron (MLP) given the identical distributional input, and evaluate each at two grid resolutions on two metrics: the household-optimality (Euler) residual, and the unbiased bond-market-clearing residual that no method is trained on (Table 6). On both metrics the distribution-conditioning methods attain residuals roughly an order of magnitude below moment-KS, and the gap persists as the grid refines. Figure 8 shows the operator’s recovered consumption policy over the liquid–illiquid plane; the operator conditions on the full joint (b, a) shape rather than its marginals.

Table 6: Two-asset HANK: ergodic residuals by method and grid (failure regime).

Method	Euler residual (median)		Bond-clearing residual	
	$b=a=24$	$b=a=40$	$b=a=24$	$b=a=40$
Moment-KS	2.30%	1.33%	2.63%	1.83%
Operator (FNO)	0.23%	0.10%	0.18%	0.20%
NN (MLP)	0.28%	0.18%	0.19%	0.21%

The Euler residual is the metric the operator and MLP are trained on, and so favors them; the bond-clearing residual $|E[b'] - \bar{B}|/\bar{B}$ is unbiased. Part of the clearing gap is representational, in that the distribution-conditioning policies are smooth in R^b whereas the moment-KS argmax is a discrete-grid step, and the gap narrows as the moment-KS grid refines.

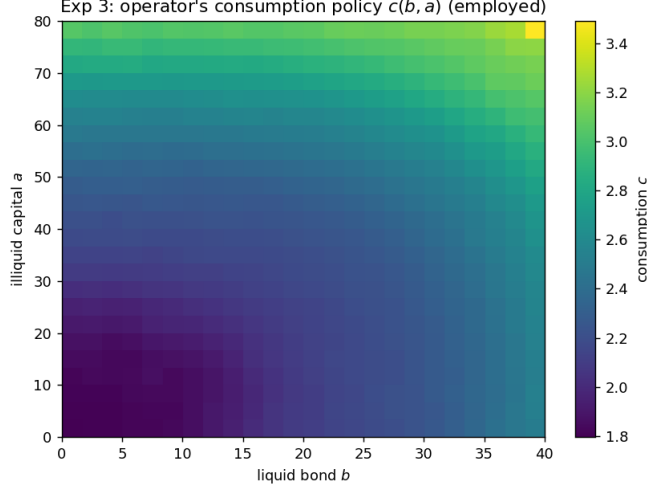


Figure 8: Two-asset HANK: the operator’s consumption policy $c(b, a)$ over the liquid–illiquid plane (employed), rising in both assets and lowest at the constrained corner.

Isolating the distributional information. A further test identifies what the operator uses. DeepHAM (Han and Yang, 2021) compresses the distribution into a small number of learned generalized moments and conditions the policy on those. On this economy it matches the operator on the trained Euler metric (0.31%) but leaves a 31% residual on the neutral bond-clearing metric, and widening the fixed bottleneck did not help within a stable-training regime (a compact variant left 36%; a wide one became unstable and collapsed the ergodic capital). We read this as evidence that a small, fixed learned-moment bottleneck is fragile here, rather than as a proof that no learned-moment count could succeed; conditioning on the distribution as a function sidesteps the choice of bottleneck. Two caveats temper the comparison: the operator and MLP are trained on the Euler metric, and part of the clearing gap is representational (a smooth policy against a discrete-grid argmax).

6.4 Decreasing-returns heterogeneous firms

Our second failure economy replaces households with a continuum of firms that draw idiosyncratic productivity z and produce with decreasing returns to scale, in the specification of Azinovic-Yang and Žemlčka (2025) building on Khan and Thomas (2008). The market-clearing wage then depends on the dispersion of the firm distribution: aggregate labor demand integrates a function convex in productivity, so by Jensen’s inequality the wage is a nonlinear, shape-dependent functional of the distribution that the mean cannot pin down, and approximate aggregation fails for a theoretical, calibration-free reason. A Bloom et al. (2018)-style uncertainty regime, in which the dispersion of idiosyncratic productivity itself shifts over time, makes the wage-relevant shape of the firm distribution vary along the

ergodic path; without it the distribution would sit nearly static, as in Sections 6.1–6.2.

Figure 9a forecasts the clearing wage from the firm distribution under this regime. A naive moment rule (mean capital, aggregate productivity, and the uncertainty state) misses the dispersion-driven movement in the wage ($R^2 \approx 0.86$), while a rich moment set and the operator both recover it ($R^2 \approx 1.0$). The operator thus performs automatic feature selection: it attains the accuracy of a well-specified moment rule without the modeler hand-picking which moments to track, and it does so by conditioning on the whole distribution rather than a fixed moment set, so it is not exposed to the mis-specification a fixed rule would suffer off the ergodic set. Figure 9b shows the operator’s recovered firm investment policy $k'(k, z)$: increasing in capital and shifted up by productivity, crossing the 45° line at each type’s target capital.

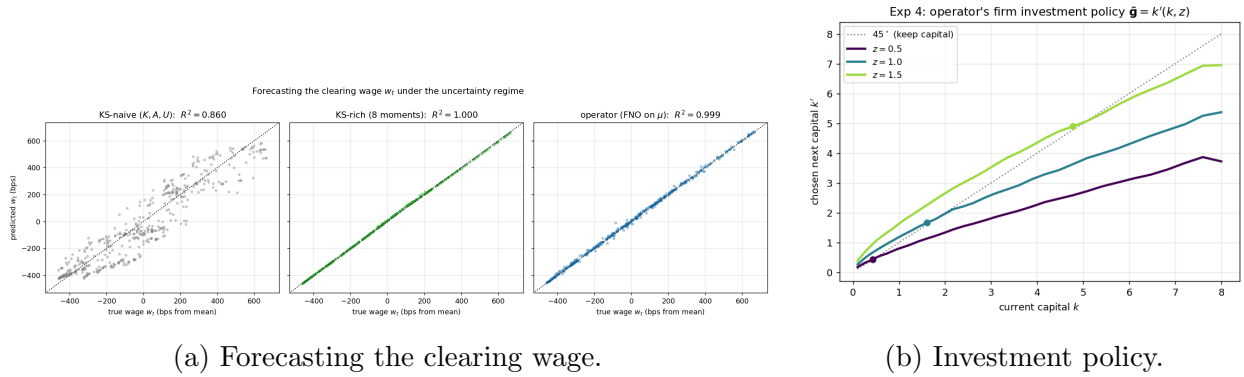


Figure 9: Decreasing-returns firm economy under the uncertainty regime. Left: forecasting the clearing wage, comparing naive moment-KS ($R^2 \approx 0.86$) with a rich moment set and the operator ($R^2 \approx 1.0$). Right: the operator’s firm investment policy $k'(k, z)$, crossing the 45° line at each type’s target capital.

6.5 Synthesis

The four economies divide according to whether the shape of the distribution varies along the ergodic path. On the two validation economies, the canonical and continuous- ϵ Krusell–Smith models, approximate aggregation holds, the shape is nearly static, and a well-specified moment method is already accurate; the operator matches it, and its contribution is to render the distributional state tractable and to keep the simulated economy stable where a generic feed-forward network diverges (Section 6.1). On the two failure economies, the two-asset HANK and decreasing-returns firm models, the shape varies and moments cannot recover the distribution-dependent price. There the operator recovers the shape-dependent liquid return and clearing wage that moment truncation and moment-bottleneck networks miss, and it does so by automatic feature selection, matching a rich, well-specified moment solution without the modeler choosing which moments to track, while conditioning on the whole distribution rather than a fixed moment set. The operator is therefore not a uniformly

more accurate solver. It is a structured and assumption-light way to carry the distribution as a genuine state variable, and its advantage is largest where the distribution’s shape is a price-relevant object.

7 Conclusion

This paper develops and evaluates an operator-learning framework for dynamic heterogeneous-agent models with aggregate risk. Rather than compressing the cross-sectional distribution into a finite, modeler-chosen object (a fixed set of moments, a panel of simulated agents, a histogram, or a fixed number of learned moments), the framework reformulates the policy as an operator that conditions on the distribution as a function, parameterized by a Fourier neural operator. No finite summary of the distribution is fixed at the input in advance; the method learns its own summary in a spectral basis whose resolution is a refinable network hyperparameter independent of the discretization, and inherits permutation invariance, aggregation sharing, and discretization invariance as consequences of that choice.

The empirical contribution is an assessment of where the reformulation helps, organized by whether the shape of the distribution varies along the ergodic path. On two validation economies, the canonical [Krusell and Smith \(1998\)](#) model and its continuous-shock extension, approximate aggregation holds and the shape is nearly static. There the operator solves the constrained policy to below one percent relative Euler error in minutes, carries a genuinely two-dimensional distribution at comparable accuracy, and keeps the simulated economy on a consistent ergodic path where a parameter-matched feed-forward network diverges; but it matches rather than improves upon a well-specified moment method. On two economies where approximate aggregation fails, a two-asset HANK model whose liquid return depends on the joint shape of wealth and a decreasing-returns firm model whose wage depends on the dispersion of the firm distribution, the operator’s distinctive value appears. It recovers the shape-dependent price that moment truncation and moment-bottleneck networks miss, and it does so by automatic feature selection, matching a rich, well-specified moment solution without the modeler choosing which moments to track, while conditioning on the whole distribution rather than a fixed moment set. The paper also documents the limits. On moment-friendly ergodic sets, which describe much of standard heterogeneous-agent macroeconomics, a well-specified moment method is already accurate, and part of the residual differences reflects the smoothness of the function-space representation rather than distributional information per se.

These findings delineate where the operator’s structural advantage is largest: settings in which an equilibrium object is a functional of the shape of the distribution rather than its low-order moments. The framework’s function-space conditioning should therefore be most valuable in heterogeneous-agent economies with high-dimensional or fat-tailed distributions, where moment truncation is least adequate, and extending it to such environments is a

natural direction for future work.

References

- Aiyagari, S Rao**, “Uninsured idiosyncratic risk and aggregate saving,” *The Quarterly Journal of Economics*, 1994, *109* (3), 659–684.
- Azinovic, Marlon, Luca Gaegauf, and Simon Scheidegger**, “Deep equilibrium nets,” *International Economic Review*, 2022, *63* (4), 1471–1525.
- Azinovic-Yang, Marlon and Jan Žemlčíka**, “Deep Learning in the Sequence Space,” *arXiv preprint arXiv:2509.13623*, 2025.
- Bloom, Nicholas, Max Floetotto, Nir Jaimovich, Itay Saporta-Eksten, and Stephen J. Terry**, “Really Uncertain Business Cycles,” *Econometrica*, 2018, *86* (3), 1031–1065.
- Blundell, Richard, Luigi Pistaferri, and Ian Preston**, “Consumption inequality and partial insurance,” *American Economic Review*, 2008, *98* (5), 1887–1921.
- Cagetti, Marco and Mariacristina De Nardi**, “Wealth inequality: Data and models,” *Macroeconomic dynamics*, 2008, *12* (S2), 285–313.
- Chen, Mingli, Andreas Joseph, Michael Kumhof, Xinlei Pan, Rui Shi, and Xuan Zhou**, “Deep reinforcement learning in a monetary model,” *arXiv preprint arXiv:2104.09368*, 2021.
- Chen, Tianping and Hong Chen**, “Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems,” *IEEE transactions on neural networks*, 1995, *6* (4), 911–917.
- Goodfellow, Ian, Yoshua Bengio, and Aaron Courville**, *Deep learning*, MIT press, 2016.
- Goswami, Somdatta, Aniruddha Bora, Yue Yu, and George Em Karniadakis**, “Physics-informed deep neural operator networks,” in “Machine Learning in Modeling and Simulation: Methods and Applications,” Springer, 2023, pp. 219–254.
- Haan, Wouter J Den and Pontus Rendahl**, “Solving the incomplete markets model with aggregate uncertainty using explicit aggregation,” *Journal of Economic Dynamics and Control*, 2010, *34* (1), 69–78.
- , **Kenneth L Judd, and Michel Juillard**, “Computational suite of models with heterogeneous agents: Incomplete markets and aggregate uncertainty,” *Journal of Economic Dynamics and Control*, 2010, *34* (1), 1–3.
- Han, Jiequn and Yucheng Yang**, “Deephm: A global solution method for heterogeneous agent models with aggregate shocks,” *arXiv preprint arXiv:2112.14377*, 2021.

- Judd, Kenneth L, Lilia Maliar, and Serguei Maliar**, “Numerically stable and accurate stochastic simulation approaches for solving dynamic economic models,” *Quantitative Economics*, 2011, *2* (2), 173–210.
- Kahou, Mahdi Ebrahimi, Jesús Fernández-Villaverde, Jesse Perla, and Arnav Sood**, “Exploiting symmetry in high-dimensional dynamic programming,” Technical Report, National Bureau of Economic Research 2021.
- Kaplan, Greg, Benjamin Moll, and Giovanni L Violante**, “Monetary policy according to HANK,” *American Economic Review*, 2018, *108* (3), 697–743.
- Khan, Aubhik and Julia K. Thomas**, “Idiosyncratic Shocks and the Role of Nonconvexities in Plant and Aggregate Investment Dynamics,” *Econometrica*, 2008, *76* (2), 395–436.
- Kovachki, Nikola, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar**, “Neural operator: Learning maps between function spaces,” *arXiv preprint arXiv:2108.08481*, 2021.
- Krueger, Dirk and Fabrizio Perri**, “Does income inequality lead to consumption inequality? Evidence and theory,” *The Review of Economic Studies*, 2006, *73* (1), 163–193.
- Krusell, Per and Anthony A Smith Jr**, “Income and wealth heterogeneity in the macroeconomy,” *Journal of political Economy*, 1998, *106* (5), 867–896.
- Li, Zongyi, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar**, “Physics-informed neural operator for learning partial differential equations,” *arXiv preprint arXiv:2111.03794*, 2021.
- , **Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Andrew Stuart, Kaushik Bhattacharya, and Anima Anandkumar**, “Multipole graph neural operator for parametric partial differential equations,” *Advances in Neural Information Processing Systems*, 2020, *33*, 6755–6766.
- , – , – , – , **Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar**, “Fourier neural operator for parametric partial differential equations,” *arXiv preprint arXiv:2010.08895*, 2020.
- , – , – , – , – , – , – , **and** – , “Neural operator: Graph kernel network for partial differential equations,” *arXiv preprint arXiv:2003.03485*, 2020.
- Lu, Lu, Pengzhan Jin, and George Em Karniadakis**, “Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators,” *arXiv preprint arXiv:1910.03193*, 2019.

Maliar, Lilia and Serguei Maliar, “Numerical methods for large-scale dynamic economic models,” in “Handbook of computational economics,” Vol. 3, Elsevier, 2014, pp. 325–477.

– **and** –, “Deep learning classification: Modeling discrete labor choice,” *Journal of Economic Dynamics and Control*, 2022, *135*, 104295.

– , – , **and Pablo Winant**, “Deep learning for solving dynamic economic models.,” *Journal of Monetary Economics*, 2021, *122*, 76–101.

Zheng, Stephan, Alexander Trott, Sunil Srinivasa, David C Parkes, and Richard Socher, “The AI economist: Optimal economic policy design via two-level deep reinforcement learning,” *arXiv preprint arXiv:2108.02755*, 2021.

– , – , – , **Nikhil Naik, Melvin Gruesbeck, David C Parkes, and Richard Socher**, “The ai economist: Improving equality and productivity with ai-driven tax policies,” *arXiv preprint arXiv:2004.13332*, 2020.